

**SELECT id FROM sift1M ORDER BY
l2distanceapproximate(embedding, query) LIMIT 10;**

**SELECT id FROM sift1M ORDER BY
l2distanceapproximate(embedding, query) LIMIT 10;**

HNSW (Hierarchical Navigable Small World, Malkov & Yashunin, 2016) 凭借在工程实践中以较小资源即可达到高召回与低延迟的能力，已成为在线高性能向量检索的事实标准。Apache Doris 自 4.0 版本起支持基于 HNSW 的 ANN 索引 (Approximate Nearest Neighbor, 近似最近邻索引)。本文从算法原理出发，结合参数与工程实践，介绍如何在 Apache Doris 生产集群中使用与调优 HNSW 索引。

快速导航

- 不熟悉 HNSW? 先阅读 在 HNSW 之前 与 Hierarchical Navigable Small World 了解算法演进。
- 想直接上手? 跳转到 HNSW In Apache Doris 查看建表、索引构建与查询示例。
- 关注效果? 参考 召回率优化 与 Benchmark。
- 关注资源开销? 参考 内存空间与性能。

在 HNSW 之前

HNSW 算法在论文 Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs 中被提出。在 HNSW 之前，业界已有许多近似 KNN 搜索算法，但都存在各自的问题。

Proximate Graph

近邻图 (Proximate Graph) 算法的基本流程：

1. 从图的某个入口 (随机顶点或策略选定顶点) 出发，迭代式遍历图。
2. 每一步计算查询向量与当前基节点所有邻居的距离，选择距离最小的邻居作为下一次的基节点。
3. 持续维护迄今为止找到的最佳候选邻居集合。
4. 当满足停止条件 (如一轮迭代未找到更近节点) 时停止，从候选集中取 Top K 作为最终结果。

近邻图算法本质上是对 Delaunay Graph 的近似，因为 Delaunay Graph 具有一个重要性质：**贪心搜索总是能找到最近邻**。

但这类算法存在两个明显缺陷：

- 缺陷：查询路由阶段的迭代次数随数据量呈幂律级别增长；影响：大规模数据下查询代价急剧上升
- 缺陷：难以构建高质量的 proximity graph，容易出现局部聚集；影响：图缺乏全局连通性，部分节点难以被路由到

图片说明：低质量 proximity graph 示意图

上图直观展示了一个不理想的 proximity graph：颜色越深的点连通性越差，部分点几乎找不到自己的邻居，搜索阶段也就难以路由到这些点。

Navigable Small World

为了解决上述问题，业界给出两种思路：

1. **混合算法**：在搜索邻接图前，先做一次粗排，找到更合适的入口点，再进行贪婪搜索。
2. **可导航小世界结构**：通过良好连通性 + 限制每个节点的最大邻居数量，控制搜索复杂度。

NSW (Navigable Small World) 采用的是第二种思路。

NSW 模型最早由 J. Kleinberg 在社会实验中提出，用于研究社会中人与人之间的联系——也就是著名的六度分割理论。

具体到 K-NN 图算法，所有在搜索时具有对数复杂度或多重对数复杂度的小世界网络都被称为可导航小世界网络 (Navigable Small World Network)，实现方案众多，此处不展开。

NSW 在部分数据集上代表了当时最先进的搜索性能，但因其并非严格的对数复杂度，在低维向量空间等测试集上表现欠佳。

Hierarchical Navigable Small World

NSW 算法在搜索时分为两个阶段：

- 阶段：**zoom-out**；行为：随机选择一个低度数顶点作为入口，搜索时偏好高度数节点，直到某节点到其邻居的平均距离超过该节点到查询向量的距离

- 阶段：**zoom-in**；行为：找到合适的高度数节点后，开始执行贪婪搜索，直到找到最佳的 TopN

NSW 之所以是多重对数复杂度，是因为总的距离计算次数大致正比于「跳跃次数 × 经过顶点的平均度」，而平均跳跃次数与平均度均与数据规模成对数关系，因此整体是多重对数复杂度。

HNSW 通过加速 zoom-out 过程将查询时间复杂度降低为对数时间复杂度。

图片说明：HNSW 分层结构示意图

HNSW 中的「分层」指：按照顶点边的典型长度范围（characteristic radius），将 NSW 图中的顶点分层。搜索流程如下：

1. 选取最高层的入口点开始。
2. 在当前层执行贪心搜索，找到该层最近点。
3. 向下移动一层，重复贪心搜索。
4. 直到最底层，得到最终结果。

由于每层节点的最大连接数有上限，整体保证了对数时间复杂度。

为了构建分层结构，HNSW 在插入每个节点时，根据**几何分布**计算其所在层数 l ，确保整体不会有过多分层。HNSW 不需要在导入数据前对数据进行 shuffle（NSW 中必须 shuffle，否则影响图质量），因为索引期间的随机层级计算本身就具备随机化效果，这也使得 HNSW 支持真正的增量更新。

HNSW In Apache Doris

Doris 自 4.0 版本起支持用户基于 HNSW 算法构建 ANN 索引。

索引构建

ANN 索引的创建有两种方式：

- 方式：建表时指定；触发时机：数据导入随 segment 创建同步构建；优点：数据导入完成即可使用索引加速；缺点：同步构建会拖慢导入速度，compaction 会重复构建索引
- 方式：CREATE/BUILD INDEX；触发时机：数据导入后异步构建；优点：不影响导入速度，便于参数调优；缺点：索引构建期间无法使用 ANN 加速

方式一：建表时指定索引

适用于明确知道索引参数、不需要频繁调优的稳定场景。

```

CREATE TABLE sift_1M (
  id int NOT NULL,
  embedding array NOT NULL COMMENT "",
  INDEX ann_index (embedding) USING ANN PROPERTIES(
    "index_type"="hns",
    "metric_type"="l2_distance",
    "dim"="128"
  )
) ENGINE=OLAP
DUPLICATE KEY(id) COMMENT "OLAP"
DISTRIBUTED BY HASH(id) BUCKETS 1
PROPERTIES (
  "replication_num" = "1"
);

INSERT INTO sift_1M
SELECT *
FROM S3(
  "uri" =
  "https://selectdb-customers-tools-bj.oss-cn-beijing.aliyuncs.com/sift_database.tsv",
  "format" = "csv");

```

方式二：CREATE / BUILD INDEX

适用于需要灵活调优、对导入性能敏感的场景。

第 1 步：建表（不带索引）并导入数据。

```

CREATE TABLE sift_1M (
  id int NOT NULL,
  embedding array NOT NULL COMMENT ""
) ENGINE=OLAP
DUPLICATE KEY(id) COMMENT "OLAP"
DISTRIBUTED BY HASH(id) BUCKETS 1
PROPERTIES (
  "replication_num" = "1"
);

INSERT INTO sift_1M
SELECT *
FROM S3(

```

```
"uri" =
"https://selectdb-customers-tools-bj.oss-cn-beijing.aliyuncs.com/sift_database.tsv",
"format" = "csv");
```

第 2 步：通过 CREATE INDEX 添加索引定义。此时仅有索引定义，存量数据上还未真正构建索引。

```
CREATE INDEX idx_test_ann ON sift_1M (`embedding`) USING ANN PROPERTIES (
  "index_type"="hnsw",
  "metric_type"="l2_distance",
  "dim"="128"
);
```

```
SHOW DATA ALL FROM sift_1M;
```

输出示例：

```
+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+
| TableName | IndexName | ReplicaCount | RowCount | LocalTotalSize | LocalDataSize |
LocalIndexSize | RemoteTotalSize | RemoteDataSize | RemoteIndexSize |
+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+
| sift_1M | sift_1M | 1 | 1000000 | 170.001 MB | 170.001 MB | 0.000 | 0.000 |
0.000 | 0.000 |
| | Total | 1 | | 170.001 MB | 170.001 MB | 0.000 | 0.000 | 0.000
| 0.000 |
+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+
2 rows in set (0.01 sec)
```

第 3 步：通过 BUILD INDEX 触发实际构建。

```
BUILD INDEX idx_test_ann ON sift_1M;
```

BUILD INDEX 是异步执行的，可通过 SHOW BUILD INDEX 查看任务状态：

```
SHOW BUILD INDEX WHERE TableName = "sift_1M";
```

输出示例：

```
+-----+-----+-----+
+-----+-----+-----+
-----+-----+-----+-----+-----+-----+-----+
```

JobId	TableName	PartitionName	AlterInvertedIndexes
CreateTime	FinishTime	TransactionId	State Msg Progress
1763603913428	sift_1M	sift_1M	[ADD INDEX idx_test_ann (`embedding`) USING ANN PROPERTIES("dim" = "128", "index_type" = "hnsw", "metric_type" = "l2_distance")],
2025-11-20 11:14:55.253	2025-11-20 11:15:10.622	126128	FINISHED NULL

DROP INDEX

调优阶段可能需要测试不同参数组合，使用 DROP INDEX 灵活管理索引：

```
ALTER TABLE sift_1M DROP INDEX idx_test_ann;
```

进行查询

ANN 索引同时支持 **TopN search** 与 **Range search** 加速。

当向量列维度较高时，描述查询向量的字符串本身会引入额外解析开销。**生产环境中（尤其是高并发场景）不建议直接使用原始 SQL 执行向量搜索**，推荐使用 prepare statement 提前完成 SQL 解析。

Doris 官方提供了向量搜索 Python library，封装了基于 prepare statement 的必要操作，并集成了数据转换流程，可直接将查询结果转为 pandas DataFrame，方便基于 Doris 开发 AI 应用。

```
from doris_vector_search import DorisVectorClient, AuthOptions
```

```
auth = AuthOptions(
    host="localhost",
    query_port=9030,
    user="root",
    password="",
)
```

```
client = DorisVectorClient(database="demo", auth_options=auth)
```

```
tbl = client.open_table("sift_1M")
```

```
query = [0.1] * 128 # Example 128-dimensional vector

# SELECT id FROM sift_1M ORDER BY l2_distance_approximate(embedding, query) LIMIT
10;
result = tbl.search(query, metric_type="l2_distance").limit(10).select(["id"]).to_pandas()

print(result)
```

执行结果示例：

```
   id
0 123911
1  11743
2 108584
3 123739
4  73311
5 124746
6 620941
7 124493
8 177392
9 153178
```

召回率优化

向量搜索场景里最重要的指标是**召回率**——一切性能数据只有在满足一定召回率的前提下才有意义。影响召回率的因素主要有：

1. HNSW 索引阶段参数（max_degree、ef_construction）与查询阶段参数（ef_search）
2. 索引向量量化
3. segment 的大小与数量

本节讨论第 1 项与第 3 项的影响，向量量化在其他文章中介绍。

索引超参数

HNSW 索引以分层图组织向量。**构建阶段**主要包含三步：

1. **多层随机赋级（Layer assignment）**：每个向量被随机分配到多个层中，较高层节点更稀疏，用于快速导航。

2. **使用 ef_construction 搜索候选邻居**：每一层中，HNSW 使用最大长度为 ef_construction 的候选队列执行广度优先的局部搜索。该值越大，邻居越准确，图结构越合理，但构建时间越长。
3. **使用 max_degree 限制连接数**：每个节点的邻居数受 max_degree 限制，避免图结构过于稠密。

查询阶段包含两步：

1. **高层贪心搜索 (Coarse search)**：从入口点自顶向下在高层执行贪心搜索，快速接近目标区域。
2. **底层广度搜索 (Fine search)**：在第 0 层使用最大长度为 ef_search 的候选队列进行更全面的邻域扩展。

三个核心参数的作用与权衡：

- 参数：max_degree；作用：图中每个节点保存的双向边个数；默认值：32；增大的影响：召回率上升，内存占用上升，查询性能下降
- 参数：ef_construction；作用：索引阶段候选节点队列最大长度；默认值：40；增大的影响：图质量提升，召回率上升，构建时间变长
- 参数：ef_search；作用：查询阶段候选节点队列最大长度；默认值：32；增大的影响：召回率上升，距离计算次数增加，查询延迟和 CPU 开销上升

下表为 SIFT_1M 数据集上的实测结果：

- max_degree: 32; ef_construction: 80; ef_search: 32; recall@1: 0.955; recall@100: 0.75335
- max_degree: 32; ef_construction: 80; ef_search: 64; recall@1: 0.98; recall@100: 0.88015
- max_degree: 32; ef_construction: 80; ef_search: 96; recall@1: 0.995; recall@100: 0.9328
- max_degree: 32; ef_construction: 120; ef_search: 32; recall@1: 0.96; recall@100: 0.7736
- max_degree: 32; ef_construction: 120; ef_search: 64; recall@1: 0.975; recall@100: 0.89865
- max_degree: 32; ef_construction: 120; ef_search: 96; recall@1: 0.99; recall@100: 0.94575
- max_degree: 32; ef_construction: 160; ef_search: 32; recall@1: 0.955; recall@100: 0.78745

- max_degree: 32; ef_construction: 160; ef_search: 64; recall@1: 0.98; recall@100: 0.9097
- max_degree: 32; ef_construction: 160; ef_search: 96; recall@1: 0.995; recall@100: 0.95485
- max_degree: 48; ef_construction: 80; ef_search: 32; recall@1: 0.985; recall@100: 0.85895
- max_degree: 48; ef_construction: 80; ef_search: 64; recall@1: 0.99; recall@100: 0.9453
- max_degree: 48; ef_construction: 80; ef_search: 96; recall@1: 1; recall@100: 0.97325
- max_degree: 48; ef_construction: 120; ef_search: 32; recall@1: 0.97; recall@100: 0.78335
- max_degree: 48; ef_construction: 120; ef_search: 64; recall@1: 1; recall@100: 0.9089
- max_degree: 48; ef_construction: 120; ef_search: 96; recall@1: 1; recall@100: 0.95325
- max_degree: 48; ef_construction: 160; ef_search: 32; recall@1: 0.975; recall@100: 0.79745
- max_degree: 48; ef_construction: 160; ef_search: 64; recall@1: 0.995; recall@100: 0.9192
- max_degree: 48; ef_construction: 160; ef_search: 96; recall@1: 0.995; recall@100: 0.9601
- max_degree: 64; ef_construction: 80; ef_search: 32; recall@1: 1; recall@100: 0.9026
- max_degree: 64; ef_construction: 80; ef_search: 64; recall@1: 1; recall@100: 0.97025
- max_degree: 64; ef_construction: 80; ef_search: 96; recall@1: 1; recall@100: 0.9862
- max_degree: 64; ef_construction: 120; ef_search: 32; recall@1: 0.985; recall@100: 0.8548
- max_degree: 64; ef_construction: 120; ef_search: 64; recall@1: 0.99; recall@100: 0.94755
- max_degree: 64; ef_construction: 120; ef_search: 96; recall@1: 0.995; recall@100: 0.97645

- max_degree: 64; ef_construction: 160; ef_search: 32; recall@1: 0.97; recall@100: 0.80585
- max_degree: 64; ef_construction: 160; ef_search: 64; recall@1: 0.99; recall@100: 0.91925
- max_degree: 64; ef_construction: 160; ef_search: 96; recall@1: 0.995; recall@100: 0.96165

可见达成同一召回率水平有多种参数组合可选。例如目标 recall@100 > 95% 时，满足条件的组合包括：

- max_degree: 32; ef_construction: 160; ef_search: 96; recall@1: 0.995; recall@100: 0.95485
- max_degree: 48; ef_construction: 80; ef_search: 96; recall@1: 1; recall@100: 0.97325
- max_degree: 48; ef_construction: 120; ef_search: 96; recall@1: 1; recall@100: 0.95325
- max_degree: 48; ef_construction: 160; ef_search: 96; recall@1: 0.995; recall@100: 0.9601
- max_degree: 64; ef_construction: 80; ef_search: 64; recall@1: 1; recall@100: 0.97025
- max_degree: 64; ef_construction: 80; ef_search: 96; recall@1: 1; recall@100: 0.9862
- max_degree: 64; ef_construction: 120; ef_search: 96; recall@1: 0.995; recall@100: 0.97645
- max_degree: 64; ef_construction: 160; ef_search: 96; recall@1: 0.995; recall@100: 0.96165

超参数选取实践方法：

1. 建立一张无索引的多向量列表 table_multi_index（包含 2~3 个向量列）。
2. 通过 stream load 等方式将数据导入该表。
3. 在所有向量列上通过 CREATE INDEX 与 BUILD INDEX 分别构建索引，**不同列使用不同参数组合**。
4. 索引构建完成后，分别在不同列上计算召回率，选出最合适的超参数组合。

索引覆盖的行数

Doris 内表的数据按层级组织：

- 层级：Table；含义：最顶层概念
- 层级：Tablet；含义：数据迁移与 rebalance 的基本单位，按分桶键将 Table 数据均匀分布到 N 个 tablet
- 层级：Rowset；含义：版本管理的基本单位，每次导入或 compaction 在 tablet 下新增一个 rowset
- 层级：Segment；含义：真正存储数据的文件

与倒排索引一样，**向量索引也作用在 segment 粒度上**。segment 大小由 BE 配置 write_buffer_size 与 vertical_compaction_max_segment_size 决定。在导入和 compaction 过程中，当 memtable 达到一定大小后下刷生成 segment 文件，并为该 segment 构建向量索引（每个索引列对应一个索引）。

根据 HNSW 的搜索与构建机制，**对于某组索引参数，其能够有效覆盖的数据范围是有限的**。当数据量超过阈值后，召回率就无法满足要求。

部分超参数与可覆盖 segment 行数的经验值：

- max_degree: 32; ef_construction: 160; ef_search: 96; num_segment: 1M; recall@100: 0.95485
- max_degree: 48; ef_construction: 80; ef_search: 96; num_segment: 1M; recall@100: 0.97325
- max_degree: 32; ef_construction: 160; ef_search: 32; num_segment: 3M; recall@100: 0.66983
- max_degree: 128; ef_construction: 512; ef_search: 128; num_segment: 3M; recall@100: 0.9931

通过 SHOW TABLETS FROM 可以查看表的 Compaction 状态，点击对应 URL 可查看 segment 数量。

Compaction 对召回率的影响

Compaction 有时会生成更大的 segment，导致原有索引超参数无法在新的更大 segment 上保持覆盖率。

最佳实践：在执行 BUILD INDEX 之前先触发一次 FULL COMPACTION。在充分合并过的 segment 上构建索引可同时获得：

- 召回率稳定
- 减少索引构建引入的写放大

查询性能

索引文件的冷加载

Doris 的 ANN 索引基于 Meta 开源的 faiss 实现。**HNSW 索引必须在完整图结构全部加载进内存后才能进行查询加速。**

建议在高并发查询前先执行一次冷查询，将涉及 segment 的索引文件预热到内存中，否则查询性能会显著下降。

内存空间与性能

HNSW 索引（无量化压缩）占用的内存空间约为其检索向量内存大小的 1.3 倍。

例如 128 维、1M 数据集，HNSW FLAT 索引大约需要 $128 \times 4 \times 1,000,000 \times 1.3 \approx 650$ MB。

不同规模下的内存预估：

- dim: 128; rows: 1M; 预估内存: 650 MB
- dim: 768; rows: 10M; 预估内存: 48 GB
- dim: 768; rows: 100M; 预估内存: 110 GB

为保证查询性能，**BE 节点需配置足够内存**，否则索引频繁 IO 会导致查询性能大幅衰减。

Benchmark

测试硬件：16C 64GB 机器；测试框架：VectorDBBench；压测客户端：另一台 16C 机器。

Doris 生产集群的典型部署模式为 FE 与 BE **分开部署**（需要两台 16C 64GB 机器）。下表同时列出了典型部署与 FE/BE **混合部署** 的测试结果。

Performance768D1M

测试命令：

```
NUM_PER_BATCH=1000000 python3.11 -m vectordbbench doris --host 127.0.0.1 --port 9030 --case-type Performance768D1M --db-name Performance768D1M --search-concurrent --search-serial --num-concurrency 10,40,80 --stream-load-rows-per-batch 500000 --index-prop max_degree=128,ef_construction=512 --session-var hnsw_ef_search=128
```

测试结果对比：

- Doris (FE/BE 分离) : max_degree=128, ef_construction=512, hnsw_ef_search=128; Doris (FE/BE 混合) : max_degree=128, ef_construction=512, hnsw_ef_search=156
- Doris (FE/BE 分离) : 0.9931; Doris (FE/BE 混合) : 0.9929
- Doris (FE/BE 分离) : 10, 40, 80; Doris (FE/BE 混合) : 10, 40, 80
- Doris (FE/BE 分离) : 163.1567 (10) 606.6832 (40) 859.3842 (80) ;
Doris (FE/BE 混合) : 162.3002 (10) 542.3488 (40) 607.7951 (80)
- Doris (FE/BE 分离) : 0.06123 (10) 0.06579 (40) 0.09281 (80) ;
Doris (FE/BE 混合) : 0.06154 (10) 0.07351 (40) 0.13093 (80)
- Doris (FE/BE 分离) : 0.06560 (10) 0.07747 (40) 0.12967 (80) ;
Doris (FE/BE 混合) : 0.06726 (10) 0.08789 (40) 0.18719 (80)
- Doris (FE/BE 分离) : 0.06889 (10) 0.08618 (40) 0.14605 (80) ;
Doris (FE/BE 混合) : 0.06154 (10) 0.07351 (40) 0.13093 (80)